Coding Assignment For Category B

Question 1

The Hostel Laundry Queue

Task

Design and implement a laundry booking system for a hostel. Students can start a wash on any free washing machine. A machine can run only one wash at a time, so it cannot be used while a wash is in progress. Different wash types have different durations and charges: Quick (30 min, ₹20), Normal (45 min, ₹30), and Heavy (60 min, ₹45). Once a wash cycle is completed, the machine must become free for the next student.

Your solution should:

1. Identify key classes (e.g., Student, WashingMachine, WashCycle) and their attributes and operations.
2. Model the relationships between these classes, including multiplicity and any inheritance or interface relationships needed for the different wash types.
3. Implement the core workflows for starting a wash, calculating the charge, and completing a wash cycle.
4. Ensure that a new wash type (e.g., Delicate) can be added without modifying the common wash booking logic.

Sample Input: Asha starts a Quick wash on Machine M1. Ravi attempts to start a Heavy wash on Machine M1. Ravi starts a Heavy wash on Machine M2. Machine M1 completes its cycle. Neha starts a Normal wash on Machine M1.

Sample Output (expected behavior): Quick wash started on M1 for Asha (30 min). Charge: ₹20.00. Machine M1 is currently busy. Heavy wash started on M2 for Ravi (60 min). Charge: ₹45.00. M1 cycle completed. M1 is now free. Normal wash started on M1 for Neha (45 min). Charge: ₹30.00.

Instructions

Consider using an abstract class or an interface for the wash type, so that each type defines its own duration and charge. The WashCycle object should track the student, the machine, and the wash type. Think about how the WashingMachine keeps track of whether it is free or busy, and make sure no outside code can change that status directly.

Starter code

```
// Implement: The Hostel Laundry Queue
```

Question 2

The Assignment Submission Portal

Task

Design and implement an assignment submission portal for a college. Students submit their work for an assignment that has a due date and maximum marks. Each submission has a status (Submitted or Graded). A submission can be graded only after it has been submitted, and once it is Graded, the student cannot resubmit it. Different types of assignments apply different late-submission penalties: a Coding assignment loses 10% of the awarded marks for each day late, and a Written assignment loses 20% for each day late. The final marks are calculated at the time of grading.

Your solution should:

1. Identify the main classes, such as Student, Assignment, and Submission.
2. Define their attributes and methods, ensuring appropriate access control so that the submission status cannot be changed from outside.
3. Model the relationships between these classes, including any generalization/specialization for different assignment types and their penalty rules.
4. Implement the core functionality for submitting work, grading it with the late penalty applied, and blocking resubmission after grading.
5. Design the system so that adding a new assignment type with a different penalty rule does not require changes to the existing grading workflow.

Sample Input: Coding assignment 'Linked List Lab' (max 50, due Mar 10) and Written assignment 'Design Essay' (max 50, due Mar 12) are created. Asha submits 'Linked List Lab' on Mar 10. Ravi submits 'Design Essay' on Mar 14. Faculty awards Asha 45 marks. Faculty awards Ravi 40 marks. Asha attempts to resubmit 'Linked List Lab'.

Sample Output (expected behavior): Asha's submission for 'Linked List Lab' received (on time). Status: Submitted. Ravi's submission for 'Design Essay' received (2 days late). Status: Submitted. Asha graded: 45/50. Status: Graded. Ravi graded: 24/50 after 40% late penalty. Status: Graded. Cannot resubmit: 'Linked List Lab' has already been graded.

Instructions

Consider using an abstract class or interface for Assignment with a method that applies its own late penalty, so each assignment type implements its specific rule. The Submission class should hold the student, the assignment, the submission date, and the status, and guard against invalid changes. Think about where the number of late days should be calculated and which class is responsible for it.

Starter code

```
// Implement: The Assignment Submission Portal
```

Question 3

The Campus Premiere Ticket Counter

Task

Design and implement a ticket booking system for the college auditorium's movie shows. A customer can book one or more seats for a show, up to a maximum of 6 seats per booking. A seat cannot be booked twice for the same show. Different seat categories have different prices: Regular ₹150, Premium ₹250, and Recliner ₹400. The booking total is the sum of the prices of the selected seats. A booking can be cancelled only before the show starts, and cancelling it must release all its seats for other customers.

Your solution should:

1. Identify the core classes such as Customer, Show, Seat, and Booking, and their attributes and operations.
2. Define how seat availability is managed for a show and how bookings are made. 3. Model the relationships between these classes, including multiplicities and any inheritance/interface relationships to accommodate the different seat categories and their prices.
4. Implement the key functionalities: checking seat availability, creating a booking, calculating the booking total, and cancelling a booking (with the show-start rule enforced).

Sample Input: Asha books Regular seats A1 and A2 and Premium seat F5 for the 7 PM show. Ravi attempts to book seat A2 for the same show. Ravi books Recliner seat R1 for the same show. Asha cancels her booking before the show starts. Neha books seat A2 for the same show.

Sample Output (expected behavior): Booking confirmed for Asha: A1, A2, F5. Total: ₹550.00. Seat A2 is already booked for this show. Booking confirmed for Ravi: R1. Total: ₹400.00. Asha's booking cancelled. Seats A1, A2, F5 released. Booking confirmed for Neha: A2. Total: ₹150.00.

Instructions

Consider using an abstract class or interface for Seat with a getPrice() method, so each seat category defines its own price. The Booking class should hold the customer, the show, and the list of booked seats. The Show should keep track of which seats are already booked. Think about what must happen to the seats when a booking is cancelled.

Starter code

```
// Implement: The Campus Premiere Ticket Counter
```

Question 4

The FitZone Membership Desk

Task

Design and implement a membership system for a campus gym. A member buys a membership plan, and each plan calculates its fee differently from a base rate of ₹1,000 per month: Monthly (1 month, full price), Quarterly (3 months, 10% off), and Annual (12 months, 25% off). A membership has a status: Active, Frozen, or Expired. A member can check in only when the membership is Active. An Active membership can be frozen and later unfrozen. Once a membership is Expired, it cannot be frozen, unfrozen, or used for check-in.

Your solution should:

1. Identify the main classes, such as Member, Membership, and MembershipPlan. 2. Define their attributes and methods, ensuring appropriate access control for managing the membership status.
3. Model the relationships between these classes, using inheritance or interfaces for the different plans and their fee rules.
4. Implement the core functionality for buying a membership, checking in, freezing, unfreezing, and expiry, including validation of status changes.
5. Design the system so that adding a new plan (e.g., Half-Yearly) does not require changes to the existing membership workflow.

Sample Input: Asha buys a Quarterly membership. Ravi buys a Monthly membership. Asha checks in. Asha freezes her membership. Asha attempts to check in. Ravi's membership expires. Ravi attempts to freeze his membership.

Sample Output (expected behavior): Quarterly membership created for Asha. Fee: ₹2,700.00. Status: Active. Monthly membership created for Ravi. Fee: ₹1,000.00. Status: Active. Asha checked in successfully. Asha's membership frozen. Status: Frozen. Check-in denied: Asha's membership is Frozen. Ravi's membership expired. Status: Expired. Cannot freeze an Expired membership.

Instructions

Define an interface or abstract class for MembershipPlan with a calculateFee() method, and let each plan implement its own discount rule. The Membership class should manage its own status and reject invalid

changes, such as freezing an expired membership. Think about how to keep the fee rules separate from the status rules.

Starter code

```
// Implement: The FitZone Membership Desk
```

Question 5

The Campus Notice Broadcaster

Task

Design and implement a notice broadcasting module for a college. The admin posts notices, and each notice has a title and one or more target departments (e.g., CSE, ECE). A notice cannot be posted without a title or without at least one target department. Each student belongs to one department and chooses one or more preferred channels to receive notices, such as Email, SMS, or App. When a notice is posted, every student in the target departments must receive it on each of their preferred channels. Each channel delivers the message in its own way.

Your solution should:

1. Identify the primary classes: Student, Notice, NoticeBoard, and the various channel implementations.
2. Define their essential attributes and operations, including how students choose channels and how a notice is posted.
3. Model the relationships between these classes, using inheritance or interfaces to handle the different channels polymorphically.
4. Implement the workflow for validating a notice, finding the students in the target departments, and delivering the notice through each preferred channel.
5. Ensure the design allows a new channel (e.g., WhatsApp) to be added without modifying the existing posting and delivery logic.

Sample Input: Asha (CSE) prefers Email and App. Ravi (ECE) prefers SMS. Admin posts notice 'Lab Closed Tomorrow' for CSE. Admin posts notice 'Fee Deadline Extended' for CSE and ECE. Admin attempts to post notice 'Sports Day' with no target department.

Sample Output (expected behavior): Notice 'Lab Closed Tomorrow' posted to CSE. [Email → Asha] Lab Closed Tomorrow. [App → Asha] Lab Closed Tomorrow. Notice 'Fee Deadline Extended' posted to CSE, ECE. [Email → Asha] Fee Deadline Extended. [App → Asha] Fee Deadline Extended. [SMS → Ravi] Fee Deadline Extended. Cannot post notice: At least one target department is required.

Instructions

Define an interface for NotificationChannel with a send() method, and create concrete EmailChannel, SmsChannel, and AppChannel classes. The NoticeBoard should depend on the NotificationChannel abstraction, not on the concrete classes. Think about where the notice validation belongs and how the NoticeBoard finds the right students for each notice.

Starter code

```
// Implement: The Campus Notice Broadcaster
```